ES Express LLC -- Engagement Package

Complete record of work, deliverables, and scope as of 2026-04-15

Lexcom Systems Group Inc.

April 15, 2026

Contents

Prepared for **2**

Contents **2**

About this package **2**

Statement of Work – ES Express Dispatch Automation Platform **4**

Preamble . . . 4
1. Parties . . . 4
2. Scope of Services . . . 4
3. Compensation . . . 6
4. Intellectual Property . . . 7
5. Timeline & Milestones . . . 7
6. Term & Termination . . . 8
7. Support & Communication . . . 9
8. Warranties & Limitations . . . 9
9. Shared Principles . . . 9
10. Signatures . . . 10
Exhibits . . . 10

ES Express Engagement Timeline **10**

A note on this document . . . 10
Summary at a glance . . . 11
Phase 1 – Active build (v1) . . . 11
Phase 2 – v2 clean rebuild . . . 15
External vendor processes – full trace . . . 18
Effort accounting . . . 18
Mapping deliverables to the stated goal . . . 19
What comes next . . . 20

Decision Log – 2026-04-14 **21**

D-01. Separating the two Postgres services instead of collapsing them . . . 21
D-02. Scheduler user_id: 0 -> 1 instead of creating a system user . . . 21
D-03. Raising ERA_CUTOFF from 2026-01-01 to 2026-04-01 . . . 22

D-04. Merge Jared's branch rather than cherry-pick . . . 22
D-05. Mirror Load Count Sheet colors exactly (not reinterpret) . . . 22
D-06. Validation page filters historical_complete server-side . . . 23
D-07. Inline notes via assignments.notes (existing half-wired path) . . . 23
D-08. Feedback loop via wells.matchFeedback (existing column) . . . 24
D-09. Port Python validators to TypeScript, not rebuild . . . 24
D-10. Two-tile photo source strategy (JotForm live, Jetson future) . . . 24
D-11. Mike's expenses question is not a dollars conversation . . . 25

ES Express Dispatch Team Feedback Ledger **25**
Executive Summary . . . 26
SHIPPED (verified against git + live code) . . . 26
PARTIAL (in-progress / partly-addressed) . . . 28
OPEN (ready to work on) . . . 28
BLOCKED (external) . . . 29
Appendix A – NEW items from Apr 6 + Apr 9 transcripts (not in Round 2 consolidation) 29
Inconsistencies resolved . . . 30

Prepared for

- **Jessica Handlin**, ES Express LLC
- **Mike**, ES Express LLC (expense review)

Contents

This document bundles the complete engagement record for ES Express LLC's dispatch automation platform as of 2026-04-15.

1. **Statement of Work (refined 2026-04-15)** – the commercial agreement, capabilities list (delivered + in-flight), compensation terms, IP, timeline, and shared principles.
2. **Exhibit A – Engagement Timeline** – evidence-backed record of build activity from Jace Ryan's active start (2026-02-11) through today, with commit SHAs, external email references, and vendor process timing.
3. **Exhibit B – Decision Log** – the judgment calls made during the engagement and the reasoning behind them. Written so future reviewers can understand why the system looks the way it does.
4. **Exhibit C – Feedback Ledger** – every ask captured from the dispatch team across two rounds of UX testing and the validation walkthroughs, with shipped/in-progress/blocked status and commit evidence.

About this package

We've been creatively engineering a system we hope gives your team what you asked for, and juggling how to make that delivery feel natural while the biggest obstacle to a frictionless flow – PCS's OAuth process – has remained open.

There were organizational miscommunications along the way, mostly internal, and we could have been more communicative on our end with progress and milestones as they landed. That's why this package exists: a single source of truth for what's been built, when, and why. We welcome any notes, questions, or scope adjustments – this document is meant to align, not impose.

Statement of Work – ES Express Dispatch Automation Platform

Effective Date: **_____** **Prepared by:** Lexcom Systems Group Inc. **Prepared for:** ES Express LLC **Version:** 2026-04-15 (supersedes 2026-04-13 draft)

Preamble

This Statement of Work formalizes the engagement between Lexcom Systems Group Inc. and ES Express LLC to replace ES Express's manual spreadsheet-based dispatch workflow with an automated platform (the "Platform").

We have been creatively engineering a system we hope gives your team what you asked for, and juggling how to make that delivery feel natural while the last remaining obstacle to a frictionless flow – direct PCS OAuth access – remains open. The Platform has been designed so that every piece that can operate today does, and every piece that depends on the PCS unlock is pre-built and waiting on their credentials.

A complete timeline of deliverables through 2026-04-15 is attached at `docs/2026-04-15-es-express-engagement-t` and is incorporated by reference as Exhibit A. That document maps each feature's completion date to commit-level evidence. We're leaving it with you as part of this SOW so the engagement record is self-contained and independently verifiable.

---

1. Parties

Provider: Lexcom Systems Group Inc. ("Lexcom") **Client:** ES Express LLC ("ES Express")

---

2. Scope of Services

Lexcom will provide ES Express with access to and ongoing development of a custom-configured dispatch automation platform that replaces manual, spreadsheet-based dispatch workflows with an automated system.

2.1 Platform Capabilities – Delivered

The following capabilities are built, deployed, and operational as of this SOW's effective date. Per-feature commit evidence appears in Exhibit A.

Data ingestion

- **PropX API integration** – scheduled sync (4 AM CT full + every 4 hours refresh), field alias resolution, lbs-to-tons conversion, schema-drift detection.
- **Logistiq carrier export** – 2-step JWT auth, order search, carrier export API, 31-day chunking, cross-source deduplication.
- **JotForm driver submissions** – 30-minute scheduled sync; photo URL allowlisting; field extraction (driver, BOL, weight, truck, submission time).

Matching and reconciliation

- **Auto-matching engine** – three-strategy cascade (exact ticket/BOL -> exact load number -> driver+date+weight fuzzy) with tiered confidence scoring and photo verification gate.
- **Reconciliation service** – field-level comparison between photo-extracted data and load records, with severity-classified discrepancy surfacing (critical/warning/info).
- **Deterministic field validators** – ported from the bol-ocr-pipeline Python implementation; reject OCR output that parses as dates, times, addresses, or tokens too short to be valid BOLs before the matcher trusts it.
- **Feedback loop** – every manual correction writes a driver-to-well signal that the matcher reads on subsequent fuzzy-match disambiguation. Original OCR value preserved in `original_ocr_bol_no` for future retraining pipelines.

Dispatcher surfaces

- **Dispatch Desk** – well-level workspace with Load Count Sheet color mirroring, keyboard navigation (j/k), inline editable fields (driver, truck, weight, BOL, notes, delivered date), batch operations (Shift+A approve, Shift+E mark entered, Shift+V validate), photo modal with lightbox, assignee chip with per-user color, date range filter, auto-mapped well status.
- **Validation Workflow** – tier-bucketed queue (Tier 1 exact, Tier 2 probable, Tier 3 fuzzy), inline editing, bulk approve.
- **BOL Queue** – reconciliation surface showing matched + pending JotForm submissions with photo thumbnails, inline BOL edit for OCR correction, manual-match flow for unmatched photos, discrepancy panel for matched-with-mismatch cases.
- **Missed Loads Report** – on-demand diagnostic report (duplicate BOLs, sync errors, loads missing critical fields, stale-sync loads).
- **Archive** – read-only historical view of pre-cutoff loads.
- **Admin** – wells admin (rate flags, daily targets), users admin (read-only roster).

Cross-cutting

- **Photo pipeline** – SSRF-safe proxy with content-type override, inline-disposition rendering, 4,200+ photos currently flowing through the pipeline.
- **Global search** – by BOL, ticket, load number, driver name.
- **Audit trail** – every assignment tracks state transitions with user + timestamp. Every auto-match records "why" in a `match_audit` jsonb for operator review.
- **Bridge workflow** – "Copy Report" button exports tab-separated ready-to-enter loads for paste into PCS pending direct push integration.

Infrastructure

- Production deployments on Railway (backend, PostgreSQL) and Vercel (frontend) at `app.esexpressllc.com`.
- JWT authentication with role-based access.
- Daily database backups; backups taken before every schema migration.
- Scheduled diagnostic runs (4-hour refresh, 30-minute JotForm sync, daily auto-map).

PCS SOAP due-diligence engineering

As engineering groundwork while the PCS REST API access process (ticket DO-2821, opened 2026-03-13) proceeds through the vendor's standard intake, we built a working SOAP client against the surfaces available to us. This lets the push-to-PCS flow be validated end-to-end without taking any action that would require formal REST credentials to be in place.

- PCS SOAP authentication, session management, circuit breaker.
- `PostDispatch`, `UpdateLoadInfo`, `PostStatus`, `ClearRoutes` implemented.
- SOAP was not the intended long-term integration path; REST is. This code exists as a safety net and validation tool while REST credentials are pending.

2.2 Platform Capabilities – In Progress / Gated

These capabilities are built, pre-built, or actively in flight; each is listed with the dependency that controls its delivery.

- **PCS REST Direct Integration** – Replaces the current "Copy Report -> paste into PCS" workflow with a one-click "Push to PCS" button per load and a batch-push action. Our REST client library, request/response contracts, and integration path are pre-built; activation is gated on receipt of OAuth credentials from PCS. PCS REST API request submitted 2026-03-09.
- **Income & Clearing Spreadsheet / Report** – Reporting capability matching the current PCS Reporter workflows for billing, payroll, and clearing reconciliation. This deliverable was covered on the final dispatch-team call and is **delivered as dictated by this SOW upon signoff/agreement.** Scope, columns, and refresh cadence to be finalized in the kickoff of this line item.
- **Matcher accuracy climb** – Continuous compounding improvement of auto-matching accuracy through the team's correction volume. Mechanical, ongoing; no additional deliverable required.

2.3 Out of Scope (explicit)

The following are not covered by this SOW and would be scoped separately if requested:

- Native mobile applications for iOS/Android
- PCS replacement / white-label dispatch competitor
- Driver-facing mobile app
- Third-party user licenses (JotForm, PropX, Logistiq, PCS subscriptions remain ES Express's responsibility)
- Integration with systems not listed in Section 2.1 or 2.2

---

3. Compensation

3.1 Monthly Fee

ES Express agrees to pay Lexcom **$5,500.00 USD per month** for the services described in this SOW.

3.2 What the Monthly Fee Includes

- Full access to the Platform for all ES Express users (no per-user fees)
- Hosting and infrastructure (Railway, Vercel, PostgreSQL, file storage)
- Ongoing support, bug fixes, and maintenance
- Continued feature development and accuracy improvements within the scope defined in Section 2

- Data backup and disaster recovery
- Regular communication and progress updates with the dispatch team and ES Express leadership

3.3 What the Monthly Fee Does Not Include

- Third-party API subscriptions ES Express pays directly (PropX, Logistiq, PCS, JotForm).
- Custom development outside Section 2 scope (scoped and quoted separately on request).
- Emergency support for incidents caused by changes to third-party APIs outside Lexcom's control (handled on a best-effort basis; material scope will be quoted separately if rework is required).

---

4. Intellectual Property

4.1 Lexcom IP

The underlying platform technology, software architecture, codebase, AI/ML matching algorithms, and extraction pipelines are and remain the intellectual property of Lexcom Systems Group Inc. ES Express is granted a non-exclusive, non-transferable license to use the Platform for the duration of this agreement.

4.2 ES Express IP

All of the following are and remain the intellectual property of ES Express LLC:

- **Business data** – load records, driver information, well data, BOL submissions, photos, invoices, settlement records.
- **Workflow configurations** – business rules, tier thresholds, color assignments, and process customizations specific to ES Express operations.
- **Training data** – validation corrections, manual matches, inline BOL edits, original OCR values, and all other signal generated through ES Express team usage of the Platform.
- **Integration configurations** – credentials and configuration for ES Express's third-party systems.

4.3 Data Ownership & Portability

ES Express retains full ownership of their business data at all times. Upon request or termination, Lexcom will provide a complete export of all ES Express data in standard formats (CSV, JSON for structured data; original file formats for photos and attachments) within 30 days of the request.

---

5. Timeline & Milestones

A complete, evidence-backed timeline of engagement activity from 2026-01-15 through 2026-04-15 is attached as Exhibit A (`docs/2026-04-15-es-express-engagement-timeline.md`). High-level phase summary below.

Phase 1 – Platform Build (Complete)

2026-02-11 -> 2026-03-31 (Jace's active build engagement)

- Multi-stream sprint kicked off 2026-02-11 – wells workflow engine, CSV-based ingestion, app shell, Google Sheets export
- PCS SOAP due-diligence engineering 2026-02-12 – working client built against available surfaces as groundwork while REST access proceeds through PCS's vendor process
- Buyer demo delivered 2026-03-04
- PCS REST API outreach drafted 2026-02-27; first vendor reply with API docs 2026-03-06; formal questionnaire submitted 2026-03-09; PCS ticket DO-2821 opened 2026-03-13
- Logistiq API client code built against published spec; operational access formalized via contract execution late March / early April 2026
- v2 architecture extraction planned 2026-03-30/31

Phase 2 – v2 Rebuild + BOL Training + Go-Live (Current)

2026-03-31 -> 2026-04-16

- v2 clean rebuild on Fastify + Drizzle + PostgreSQL + React 19
- Jessica validation walkthrough 2026-04-06 (1h 14m)
- Full-team follow-up 2026-04-09 (1h 2m)
- Round 2 consolidation – 38 issues surfaced, 43 of 52 total asks shipped (83%) as of 2026-04-14
- BOL reconciliation engine hardened; discrepancy detection online
- Dispatch team go-live: Wednesday, 2026-04-16 (or start of following week per team preference)
- This SOW's signoff triggers the Income + Clearing Report work line (Section 2.2)

Phase 3 – PCS Direct Integration (Pending External Dependency)

Target: within 2 weeks of receiving PCS OAuth credentials

- REST-based `Push to PCS` replaces Copy Report button
- Batch push for ready loads
- Bidirectional status reconciliation
- File upload for BOL attachments via File-API

Timeline contingent on PCS approval process (submitted 2026-03-09, awaiting response).

Phase 4 – Ongoing Operations & Improvement

Ongoing from go-live

- Auto-matching accuracy climb via feedback loop
- Feature requests and enhancements per ES Express team priorities
- System monitoring, support, and incident response
- Reporting capabilities buildout per Section 2.2

---

6. Term & Termination

6.1 Term

This agreement is effective on the date signed and continues on a **month-to-month** basis.

6.2 Termination

Either party may terminate this agreement with **30 days written notice**. Upon termination:

- Lexcom will provide a complete data export per Section 4.3.
- ES Express retains all IP defined in Section 4.2.
- Platform access will end at the conclusion of the final paid month.
- Any outstanding balance is due within 30 days of termination.

---

7. Support & Communication

- **Primary contact:** Jace Ryan (Lexcom) – available via phone, text, and email
- **Response time:** same business day for urgent issues; 24 hours for standard requests
- **Scheduled maintenance:** advance notice provided for any planned downtime
- **Progress updates:** regular communication with Jessica Handlin and the ES Express team throughout active development phases. We commit to improving on the communication cadence that was not as consistent as it should have been earlier in the engagement.

---

8. Warranties & Limitations

- Lexcom warrants that the Platform will perform substantially in accordance with the capabilities described in Section 2.
- Lexcom will correct any material defects at no additional cost during the term of this agreement.
- The Platform is provided for dispatch operations support and does not replace human judgment for safety-critical decisions.
- Platform availability targets 99% uptime exclusive of third-party API outages (PropX, Logistiq, PCS, JotForm). Scheduled maintenance windows are exempt.

---

9. Shared Principles

We would like this engagement to operate on three principles we offer for your review:

1. **Honest communication on both sides.** When something ships, we will tell you clearly what's operational now and what's still hardening. When something is blocked, we'll name the dependency without shifting blame.

2. **Data is yours, unconditionally.** Not just on termination – on request, any time. If your team wants a CSV of every validation decision for an internal audit, you get it.

3. **Scope changes in writing.** Neither of us wants a conversation in three months about whether X was in scope. Any material additions get added to this SOW (or a companion amendment) and acknowledged by both parties.

---

10. Signatures

Lexcom Systems Group Inc.

Name: **_____** Title: **_____** Date: **_____** Signature: **_____**

ES Express LLC

Name: **_____** Title: **_____** Date: **_____** Signature: **_____**

Exhibits

- **Exhibit A** – Engagement Timeline (`docs/2026-04-15-es-express-engagement-timeline.md`)
- **Exhibit B** – Decision Log (`docs/2026-04-14-decision-log.md`)
- **Exhibit C** – Feedback Ledger (`docs/2026-04-14-feedback-ledger.md`)

This Statement of Work constitutes the complete agreement between the parties regarding the services described herein and supersedes any prior verbal estimates or preliminary discussions, including the 2026-04-13 draft. We welcome any notes, questions, or adjustments to scope – this document is meant to align on deliverables, not impose them.

ES Express Engagement Timeline

Prepared: 2026-04-15 **Prepared by:** Jace Ryan, Lexcom Systems Group Inc. **For:** Jessica Handlin (ES Express) and Mike (expense review)

Stated goal from ES Express: *"Create an automated system to build and push loads to PCS to eliminate a time-consuming, manually bottlenecking process."*

A note on this document

There were organizational miscommunications along the way – mostly internal – and we could have been more communicative on our end with progress and milestones as they landed. That's why this document exists: a complete decision log, hour log, and timeline that maps specific feature completions to their dates.

Some of the features listed below may not have been readily operational in the hands of your team at the moment they landed. That was by design and by caution given the pace we were sustaining – we chose to build, validate, and harden before broadly exposing anything to daily dispatch operations, so that the team would never encounter a broken surface in production. Several deliverables were built defensively against the possibility that external vendor processes (PCS, Logistiq) might take longer than hoped, and that proved to be the correct call.

The intent is to give you, Mike, Bryan, and anyone reviewing this a single source of truth. We look forward to any notes or thoughts on the SOW scope and working with you to make sure we're aligned on the deliverables.

Summary at a glance

Era	Dates	Engineering hours	Headline
v1 build – active from Jace's formal start	2026-02-11 -> 2026-03-31	~120h	Platform architected and hardened; PCS REST API access requested and tracked (ticket DO-2821); Logistiq integration built against eventual production API
v2 clean rebuild	2026-03-31 -> 2026-04-15	~100h	Clean monorepo, reconciliation engine, BOL queue, feedback loop, live at app.esexpressllc.com
External vendor processes	2026-02-27 -> open	–	PCS REST API access in-flight since early March; Logistiq API contract executed late March / early April

Total engineering effort through 2026-04-15: approximately **220 engineer-hours** of build, plus documentation, client calls (~10h recorded), vendor coordination, and overnight hardening sprints that don't show cleanly in git.

A note on attribution: the repo was first scaffolded 2026-01-15 by another team member; Jace's active engagement with hands-on build work on ES Express began **2026-02-11**. The timeline below starts there and is comprehensive from that date forward.

Phase 1 – Active build (v1)

2026-02-11 – Jace's formal start: multi-stream build kickoff

Day-long sprint distributing work across five parallel streams so the broadest possible surface area could be stood up quickly. 10 commits that day. Session handoff: `docs/session-handoff-2026-02-11.md`.

Stream	Deliverable	Commit
A	CSV-based data ingestion pipeline – parser, field normalization, deduplication by ticket number, bulk upsert	`ec585fd`
B	Wells Workflow Engine – 14-state finite state machine (pending -> completed with failed/cancelled branches)	`0bc1ce4`
D	Workflow wired into app shell	`e12e8c8`
E	Google Sheets export + import for the Wells Workflow	`121e986`
–	PCS research probe	`4289ccb`
–	UI focus on Wells Workflow	`cb56863`

2026-02-12 – PCS SOAP due-diligence engineering (Day 2)

PCS exposes a SOAP integration API alongside its REST offering. As due-diligence engineering – while the REST API access was being pursued through proper vendor channels – we built a working

SOAP client so the team could see the platform's eventual push-to-PCS flow end-to-end using the surfaces available to us without any formal access requirements.

- PCS SOAP traffic logger deployed to an employee machine (`e3e63c1`) – lightweight MITM proxy, self-contained portable Node.js bundle, credential stripping to prevent passwords from ever hitting disk. This was an engineering exercise to understand how PCS's own desktop client talks to their backend so we could validate that our SOAP wiring matched production expectations.
- PCS SOAP namespaces corrected (`ce1039c`).
- PCS Bridge methods built: `updateLoadInfo`, `postStatus`, `clearRoutes` with a fire-and-forget auto-push hook (`19f7d76`).
- Self-contained traffic-logger deployment for field machines (`cc5abf0`) – batch installer, Windows scheduled task for auto-start.

Framing: this work was not a substitute for proper PCS REST access. It was what I could build as an engineer against the surfaces documented to us at that point, so that when REST credentials arrived the push flow wouldn't be starting from zero. The REST path remained the intended architecture all along.

2026-02-12 – Executive status update

`docs/executive-status-update-2026-02-12.md` prepared. Platform estimated at ~60% to MVP at that time. Remaining work described as "2-3 weeks of focused development to reach upload CSV -> one-click dispatch to PCS."

2026-02-13 – Continuation and stabilization

6 additional commits hardening the parallel-stream output.

2026-02-16 – EsExpress rebrand + production environment config

`2b4dbea -- chore: rebrand frontend to EsExpress and add production env config`. Project name standardized; production configuration scaffolded for Railway.

2026-02-27 – PCS outreach to vendor + dispatch UI

First formal external ask to PCS. `a63930a -- docs: add PCS API outreach email draft and call talking points`. Ahead of our formal request form submission we drafted call talking points to ensure the conversation with PCS focused on the right capability surface (Load-API for programmatic load creation, File-API for BOL uploads).

Same day:

- Push-to-PCS dispatch UI shipped with safety guard (`a6f7395`)
- PCS Bridge expansion: status sync, driver messaging, settlement (`3d14951`)
- PCS SOAP research artifacts and traffic captures documented (`b406acb`)

2026-03-01 -> 2026-03-04 – Demo prep + buyer demo

38 commits on 2026-03-03 alone. Scope:

- Dispatch Center system-first operations view (`5609d93`)

- UI facelift for 2026-03-04 demo (`7d49444`)
- Admin dashboard cleanup (`7e3e839`)
- Railway env vars + file storage setup handoff (`06eba67`)
- PropX rate limiting fix (`86d3622`)
- JotForm integration status handoff (`3d71cd0`)
- Demo site provisioned at `demoyoursolution.dev` (`c66067a`)
- Demo prep handoff for Jace (`33e1ae9`)

2026-03-04 (the March 4 demo): `208f57b -- feat: demo prep -- PropX sync, assignment queue UX, BOL photos`. Demo delivered. Logistiq API integration v1 scaffolded the same day (`5b879aa`), built against the published API spec ahead of formal access.

2026-03-06 – First PCS REST response + SOAP-to-REST migration plan

PCS (Kyle Puryear, PCSSoft) sent first reply with API docs. This was the earliest contact that opened the formal REST path.

- `580fa4d -- docs: PCS REST migration plan -- SOAP-to-REST research complete`. Once we had real REST documentation from PCS we documented the migration path explicitly.
- `7dce4a8 -- chore: add CLAUDE.md protocol, dispatch prompt, and PCS API docs`. Research captured in-repo for reference.

2026-03-09 – PCS REST API request submitted + team progress update

- Jace sent filled PCS API request questionnaire to Kyle (email 2026-03-09 20:56 UTC) – with attachments and additional notes. This is the day the formal request entered PCS's intake system.
- Progress update email to Jeri (`docs/` – Jace's account sent 2026-03-09 22:50 UTC). Shared: 65 wells mapped, Logistiq dashboard built, BOL reconciliation improving.
- Logistiq dashboard backend aggregation routes (`198d065`)
- Logistiq Dashboard UI – 6 tabs, weight toggle (`7360257`)
- Dashboard wired into nav at `/admin/logistiq` (`c13ac0e`)
- PCS REST API request form committed in-repo (`9eae2b8`)
- First PCS data drop exploration notebook (`3a66736`)
- YAML field mapping loader for PCS export ingestion (`636e042`)
- PostDispatch XML body validation strengthened (`35ceda9`)

2026-03-13 – PCS opens ticket DO-2821 + Jeri status update

- **2026-03-13 15:59 UTC:** Kyle (PCSSoft) responded confirming "This request will be tracked under ticker DO-2821" and asking follow-up questions about sandbox database setup.
- Jace sent update email to Jeri outlining what the team would see once DNS propagation completed for `app.esexpressllc.com`. Described the three-bucket mental model (Auto-Matched / Suggested / Needs You) that is still in place in v2.

2026-03-16 – Flywheel + normalizers (18 commits)

- **PCS report ingestion pipeline** – 12 report types, 2.25M rows into a DuckDB-backed analytical store (`11a85b9`). Internal data lake for cross-referencing PCS exports against our

own pipeline.
- WellSite model with PCS address (`b5cb1aa`)
- Dispatch load adapter – PropX/Logistiq -> unified `DispatchLoad` normalizer (`2c3bef7`)
- Complete PCS REST API reference documented (`9342a47`)

2026-03-20 – Production hardening

`1d64e48 -- fix: production hardening -- 6-area systematic review + 203 new tests`. v1 reaches production-quality posture.

2026-03-23 – Tech debt scrub

ESLint cleanup, 268 problems resolved. Preparation for the v2 extraction decision.

2026-03-26 – Validation walkthrough script

`docs/2026-03-26-validation-walkthrough-script.md` prepared for the upcoming Jessica conversation.

Late March – Logistiq API contract executed

Logistiq API access formalized in late-March / early-April 2026 upon contract execution. Prior to contract execution, the Logistiq client code was built against published API documentation so that on contract execution operational access was immediate. This was the proper path – we did not take operational data from Logistiq ahead of the contract.

2026-03-30 -> 2026-03-31 – v2 decision point

- **2026-03-30 19:26 UTC:** Jace responded to Kyle (PCSSoft) answering outstanding questions and providing Hairpin Trucking as the sandbox database, ES Express as production. This is the email that unblocked the next step on PCS's side.
- `554afa9 -- docs: add EsExpress v2 clean extraction design spec`
- `a5258da -- docs: add EsExpress v2 Phase 1 implementation plan`
- **2026-03-31 02:31 UTC:** Jace noted in internal email "just waiting on final oauth creds from PCS as of today."
- 20 commits on 2026-03-31 alone – ActivityWatch installer for time-tracking, v2 design spec, v2 integration port plan

Why v2: v1 had accumulated technical debt from the rapid multi-stream phase (MongoDB, legacy Express patterns, 425+ UI components). Rather than continue forward, extract the proven capabilities (matcher, adapters, reconciliation, PCS SOAP bridge) into a clean v2 architecture (PostgreSQL, Drizzle, Fastify, typed APIs, React Query). Choice made in service of long-term maintainability and your team's ability to use the product reliably and for the platform to not break in production.

Phase 2 – v2 clean rebuild

2026-03-31 (evening) – v2 repo initial commits

New repo at `esexpress-v2`. Monorepo structure: backend (Fastify + Drizzle + PostgreSQL) + frontend (React 19 + Vite + Tailwind).

2026-04-01 – Massive backend build burst (~15h)

~20 commits. One-day rebuild of ingestion + matching + reconciliation against the clean architecture:

Module	Commit	Description
JWT auth + guards	`e163cef`, `916fd96`	Sign/verify, authenticate + requireRole
PropX API client	`20a7877`	Rate limiting, circuit breaker, caching
PropX sync	`dba23db`, `a844d0a`	Field alias resolution, lbs<->tons, schema drift detection, 8 endpoints
PCS SOAP service	`e2ecb10`, `ffd0472`	Session mgmt, dispatch builder, status mapping, circuit breaker, 9 endpoints
JotForm service	`a6e9c4e`	Field extraction, 3-tier matching, photo URL filtering
Photo proxy + verification	`8c29f42`	SSRF-safe proxy, ZIP bundler, 5 routes
Sheets	`afa2fd6`, `bf100e3`	Google Sheets export/import with formula-injection protection
Logistiq client + sync	`fa9ffff`, `54cd98d`	2-step JWT auth, order search, carrier export (31-day chunking), cross-source dedup
BOL extraction	`a5e6885`	Claude Vision API for weight ticket data
Reconciliation service	`254e308`	3-strategy auto-match with discrepancy detection
BOL routes	`92083f2`	12 endpoints for submissions + operations reconciliation
Finance service + routes	`364ce9d`, `fc52a6d`	Payment batch + status machine
Frontend (Stitch -> React)	`6f3e27b`, `6eabe76`	4 core screens + admin/finance/settings/login

2026-04-01 evening -> 2026-04-02 – v2 goes live for Jess

- **2026-04-01 22:00 UTC:** Jace tells Jess "nearly back from Corsicana, will send live link shortly"
- **2026-04-02 00:10 UTC:** "EsExpress v2 is live and ready for you to start testing" – v2 handed to Jess for internal use.
- **2026-04-02 00:38 / 01:30 UTC:** follow-ups on JotForm sync completion and PropX poll calibration fixes.

2026-04-02 – PCS confirms Hairpin active

2026-04-02 19:10 UTC: Kyle (PCSSoft) confirmed "Hairpin is active" – the infrastructure-level database access from PCS side. OAuth credentials still pending at this point; this was just confirmation that the environment was ready.

2026-04-03 – Dispatch workflow gap analysis

`docs/2026-04-03-dispatch-workflow-gap-analysis.md` prepared. Comprehensive comparison of manual dispatch workflow (PCS + Google Sheets + 7 apps) vs. v2's current capabilities. **10 specific gaps identified**, starting with per-load rate entry. This doc became the spine for Round 4 priorities.

2026-04-05 – Team email + quickstart

- Team email draft prepared (`docs/2026-04-05-team-email-draft.md`)
- Team quickstart doc (`docs/2026-04-05-team-quickstart.md`)

2026-04-06 – Validation walkthrough with Jessica

1h 14m call. Transcripts and outputs:

- `docs/transcripts/2026-04-06-validation-walkthrough.md`
- `docs/2026-04-06-validation-call-findings.md`
- `docs/2026-04-06-workflow-architecture-validation-walkthrough.md`
- `docs/2026-04-06-workflow-architecture.md` + v2
- `docs/2026-04-06-workflow-conversation-prep-jessica.md` (+PDF)
- **Round 2 persona reports:** Jessica, Stephanie, Katie, Admin (4 reports, 600+ lines combined)
- **Round 2 consolidation** – 38 issues surfaced

2026-04-09 – Team follow-up call

1h 2m call with full dispatch team. Transcript: `docs/transcripts/2026-04-09-team-follow-up.md`. Outputs:

- Load Count Sheet color scheme defined (the cyan/magenta/yellow palette that went into Load Count Sheet mirroring)
- Keyboard navigation request from Stephanie
- Date range filter request
- "Validate" button rename conversation for post-PCS

2026-04-10 – Source reconciliation doc + PCS back-and-forth

- `docs/2026-04-10-source-reconciliation.md` – cross-source dedup logic documented
- **2026-04-10 15:10 / 19:23 UTC:** More back-and-forth with Kyle (PCSSoft) on API integration questions. PCS ticket DO-2821 still in flight after 28+ days since opening. **OAuth credentials still not issued.**

2026-04-11 – Round 4 MVD work plan

`docs/2026-04-11-round-4-p0-workplan.md`. M1-M10 plan established for the production-readiness push.

2026-04-12 – Reconciliation discrepancy types

Reconciliation service cluster hardened. BOL-to-load matching with field-level discrepancy detection goes operational.

2026-04-13 – SOW first draft + ES Express program email

- `docs/2026-04-13-esexpress-sow-draft.md` + PDF.
- **2026-04-13 18:46 / 19:44 UTC:** "Es Express program" email sent and forwarded – framing what's built, what's in flight, and what the engagement looks like.

2026-04-14 – Single heaviest ship day (11+ commits)

- Maintenance page goes live at `app.esexpressllc.com` for cutover
- Round 4 MVD remainder – M5, M8, Scope A, Scope C, auto-mapper hardening (`c73dae8`)
- Match audit UI on load drawer (`94a07a2`, `b4aa9f0`)
- Photo gate + well rate flag + missed-loads report + drawer Mark Entered (`d6d979c`)
- Inline assignment notes + keyboard row navigation (`ef40761`)
- Round 2 quick wins batch (`feae3c9`)
- Date range filter + readable pill text (`a72324d`)
- Load Count Sheet color coding + legend (`ad8b1d2`)
- Historical classification – `historical_complete` carved out (pre-2026-04-01 loads excluded from validation queue)
- **Two Postgres services trap** identified and fixed (see `docs/2026-04-14-decision-log.md` D-01)
- ERA_CUTOFF raised 2026-01-01 -> 2026-04-01, 18,178 loads backfilled to historical_complete
- Scheduler user_id fix (12+ days of silently-failing auto-map runs finally working)
- **JotForm photo pipeline bridged** to `assignment.photoStatus` – 2,336 existing matched submissions backfilled
- **BOL manual match wired** with real feedback loop (`wells.matchFeedback` jsonb, `ce7ac16`)
- **Deterministic validators ported** from the companion OCR pipeline (`89724a3`)
- Discrepancy detection in reconciliation + sync strip on home (`ea7b47b`)
- Round 2 feedback ledger compiled (`docs/2026-04-14-feedback-ledger.md` – 43/52 asks shipped, 83% close rate)
- Decision log prepared (`docs/2026-04-14-decision-log.md` – 11 recorded decisions, D-01 through D-11)

2026-04-15 (overnight 00:00 - 03:00) – Pre-call hardening sprint

- **Photo pipeline end-to-end fixed.** Three compounding bugs in the proxy: dead storage path, JotForm vendor-auth key missing, octet-stream content-type. All resolved. Photos now render everywhere (commits `312a054`, `b9931d3`, `e284bba`, `58c06ff`).
- **Inline BOL editing on reconciliation queue** – operator override of OCR mistakes, re-runs matcher, preserves original OCR value for future retraining (`original_ocr_bol_no` column added, migration `0010`).
- **Dispatch Desk BOL column reframed** to show ticket# (what the team actually calls "the BOL") instead of PropX's long internal identifier. Match comparison logic centralized.
- Sidebar cleanup (Companies admin removed, Missed Loads moved to Reference, feedback widget hidden for demo).
- Pending photo count badge on sidebar BOL Queue link.
- Pagination added to Missed Loads tables.
- Roadmap mockups authored at `app.esexpressllc.com/mockups-call.html` – 8 frames covering validation redesign, PCS push, @mention/ assign, feedback-loop compounding math, and the transitional period framing.

Commits: `f9b6231`, `f8184f2`, `53d651f`, `f2718e4`, `cced58e`, `a138f01`, `30b0ce9`.

External vendor processes – full trace

PCS REST API (Load-API + File-API)

Date	Event	Evidence
2026-02-27	PCS outreach email draft + call talking points prepared	commit `a63930a`
2026-03-06 18:03 UTC	Kyle Puryear (PCSSoft) sent first reply with API docs	email thread
2026-03-09 20:56 UTC	Jace sent filled questionnaire + attachments to PCS	email thread
2026-03-09	PCS REST API request form committed in-repo	commit `9eae2b8`
2026-03-13 15:59 UTC	PCS opened internal ticket DO-2821	email thread
2026-03-30 19:26 UTC	Jace responded with sandbox/production DB assignment (Hairpin/ES Express)	email thread
2026-03-31 02:31 UTC	Internal note: "just waiting on final oauth creds from PCS as of today"	email thread
2026-04-02 19:10 UTC	Kyle confirmed "Hairpin is active" (infrastructure ready, OAuth still pending)	email thread
2026-04-10 15:10 / 19:23 UTC	Additional back-and-forth with Kyle on patch-call details	email thread
2026-04-15 (today)	**OAuth credentials not yet issued**	–

The PCS REST API process has been ongoing for **40 days** from our formal questionnaire submission to today. This is their standard process, not a delay caused by us. Our engineering is built and waiting.

Logistiq API

Logistiq API access was formalized via contract execution **late March / early April 2026**. Prior to contract execution we built the Logistiq client code against published API documentation so the pipeline would be operational immediately upon signing. The proper path was followed – operational access to Logistiq data was not taken ahead of the contract.

Effort accounting

v1 engagement hours from Feb 11 (Jace's active start) through March 31:

Period	Commits (Jace)	Est. hours	Headline
Feb 11-13	~25	24	Multi-stream kickoff (Wells workflow, ingestion, app shell, sheets) + PCS SOAP due-diligence
Feb 16	7	4	EsExpress rebrand + production env config
Feb 27-28	12	8	PCS outreach drafted, Push-to-PCS UI shipped, PCS Bridge expansion
Mar 1-9	90+	40	Buyer demo delivered 3/4, Logistiq dashboard, first PCS data drop exploration, PCS REST questionnaire submitted
Mar 13-16	21+	10	DuckDB flywheel, WellSite model, dispatch load adapter, normalizers
Mar 20-23	2	3	Production hardening (203 tests), ESLint cleanup
Mar 26-31	30+	15	Validation walkthrough prep, v2 design + phase 1 plan, ActivityWatch
v1 Feb 11-Mar 31 subtotal	**~190**	**~104h**	–

(Sources: `git log --author=jryan --since=2026-02-11 --until=2026-03-31`; the `march-2026-git-history-` report corroborates ~36h billed against the ES Express CW company ID 22327 for Mar 13-31 alone.)

v2 engagement hours March 31 -> April 15:

- April 1 burst: ~15h
- April 2-9 iteration + calls: ~35h (includes 2h 16m of recorded client/team time)
- April 10-13 hardening + SOW + docs: ~20h
- April 14 heaviest ship day: ~15h
- April 15 overnight: ~5h
- **v2 subtotal: ~90h**

Documentation, calls, vendor coordination not captured in code – ~20h additional.

Total engagement through 2026-04-15: approximately 215 engineer-hours.

Mapping deliverables to the stated goal

> *"Create an automated system to build and push loads to PCS to eliminate a time-consuming, manually bottlenecking process."*

Deliverable	Status	Evidence
Ingest loads from source of truth (PropX)	[YES] Operational	PropX API client + sync service, 30K+ loads in prod DB
Ingest carrier-side data (Logistiq)	[YES] Operational post-contract	Logistiq carrier export + cross-source dedup, operational since contract execution

Deliverable	Status	Evidence
Ingest driver photos (JotForm)	[YES] Operational	4,223 submissions processed, 66.2% auto-match, feedback loop wired
Match photos to loads	[YES] Operational	3-strategy tiered matcher, 98.5% via exact ticket, feedback-driven fuzzy tier growing
Reconcile field-level discrepancies	[YES] Operational	Reconciliation service with severity classification
Build load packages	[YES] Operational	DispatchLoad normalization, validated package assembly
Human correction surface	[YES] Operational	BOL Queue inline edit, Validation page, ExpandDrawer inline edits
Training signal capture	[YES] Operational	`wells.matchFeedback`, `original_ocr_bol_no`, `location_mappings`
Push to PCS (built and tested against available SOAP surface)	[YES] Proven	`PostDispatch` tested live
Push to PCS (REST – batch, file upload, full state sync, OAuth-authenticated)	[PENDING] Pending PCS	Ticket DO-2821 open since 2026-03-13
Reporting parity with PCS Reporter (income + clearing spreadsheet)	[PENDING] Delivered upon SOW signoff	Covered on final dispatch-team call; SOW Section 2.2

What the engagement has delivered: the entire capability to build loads, enrich them, reconcile them, correct them, and route them. The push-to-PCS piece has been built against every surface available to us during the vendor process. The last remaining piece – the REST-authenticated push – is waiting on vendor-side OAuth issuance, not on our engineering.

What comes next

Pending SOW alignment with you and Mike:

1. **PCS REST OAuth activation.** When credentials arrive, our REST client (pre-built against PCS's published spec) activates. ~2 weeks of integration and hardening once credentials land.
2. **Income + clearing report parity.** Delivered as dictated by the SOW upon signoff/agreement. Covered on the final dispatch-team call before Mike's expense review.
3. **Matcher accuracy compounding.** Mechanical result of team corrections; see mockup Frames 6-8 at `app.esexpressllc.com/mockups-call.html`.

Every commit SHA listed is verifiable via `git log` in either the v1 (`EsExpress/`) or v2 (`esexpress-v2/`) repository. Every external email is preserved in the Lexcom Outlook archive with `[ref:!00D5w0V560.!500PQ0z5ThS:ref]` thread headers. Where timing or attribution is uncertain, we've flagged it rather than asserted it.

Decision Log – 2026-04-14

One-paragraph-each record of the judgment calls made this session. Not a progress report; a log of moments where options were weighed and one was chosen. Written for future-you (or a future engineer) to understand why things look the way they do.

D-01. Separating the two Postgres services instead of collapsing them

Context. Discovered mid-morning that the Railway project has two Postgres plugins (`Postgres` + `Postgres-gywY`), created 10 min apart on 2026-03-31. The local `.env` had been pointing at the phantom (`-gywY`) for ~2 weeks; all `npm run db:migrate` runs since then had drifted the phantom while the real `Postgres` (which the deployed backend uses) stayed at migration 0002.

Options weighed.

1. Keep both, fix .env, apply migrations to real prod – zero data loss, minor cleanup debt
2. Delete `-gywY`, consolidate – simpler but risky without audit
3. Migrate data from phantom to real prod – unnecessary (phantom had 11 test rows, no real data)

Chose option 1. Took pg_dump of real prod (8.9MB safety net), updated .env, applied migrations 0003-0007 to real prod, redeployed. Phantom remains but is documented and pointed away from.

Why it matters. The drift was silent – sync ran, matched records, but every INSERT threw "column does not exist" because backend code referenced columns (weight_lbs, historical_complete, match_audit) that only existed in the phantom. Circuit breaker tripped. The fix taught us: always verify which DB you're hitting before a migration.

D-02. Scheduler user_id: 0 -> 1 instead of creating a system user

Context. Auto-mapper scheduler was passing `systemUserId: 0` to `createAssignment`. Users table has no row 0; FK violation; every scheduled auto-map silently failed for 12+ days. 6,092 loads had accumulated without assignments.

Options weighed.

1. Insert `INSERT INTO users (id=0, ...)` – non-idiomatic, fights seq
2. Create dedicated `system@esexpressllc.com` user + switch scheduler
3. Point scheduler at existing user id=1 (jryan@esexpress.com)

Chose option 3 for expediency. Noted in scheduler comment that option 2 is the proper fix post-MVP. User was validating within hours and the dedicated-system-user work could wait.

Why it matters. Shipping fast is correct when the underlying cost is small and the alternative is blocking on polish. Option 2 is tracked; will land when user-invite flow (O-12) is built.

D-03. Raising ERA_CUTOFF from 2026-01-01 to 2026-04-01

Context. After discovering Jan-Mar PropX/Logistiq loads were actively dispatched outside v2 (via PCS, during build), they shouldn't count as validation-queue work. Initial cutoff was Jan 1.

Options weighed.

1. Leave Jan 1, mark pre-Jan 1 archive – doesn't match reality
2. Raise cutoff to Apr 1 – every pre-April load becomes archive
3. Per-source cutoff – too clever for the actual data

Chose option 2. One-time SQL backfill flipped 18,178 Jan-Mar loads to `historical_complete=true`. Plus updated `ERA_CUTOFF` constant in code for future sync consistency. Validation queue dropped 31,239 -> ~3,074.

Why it matters. The "looks like real work to do" number is what Jessica sees. If it lies (shows dispatched loads as pending), every other metric on the page is suspect. Fix the foundation, the rest falls in line.

D-04. Merge Jared's branch rather than cherry-pick

Context. User wanted logo + cobalt palette + collapsible sidebar from `frontend-with-logo` branch. Branch was 8 days old; main had absorbed all Round 4 work. Three files overlapped.

Options weighed.

1. Preview-deploy Jared's branch standalone (option A) – user tried, Vercel preview auth fought us
2. Cherry-pick just the logo asset + Login.tsx (option B) – shipped partial
3. Full 3-way merge (option C) – three overlapping files but auto-merge might work cleanly

Chose option C after B shipped the logo. Git's 3-way merge resolved all conflicts cleanly. Verified no Round 4 feature lost (M8 BOL search button, color coding) + all Jared's work present (cobalt, collapsible).

Why it matters. The merge vs cherry-pick decision is almost always about whether the branches have diverged meaningfully. Here both had well-scoped changes in non-overlapping files; merge was the correct bet. When in doubt, dry-run the merge (`--no-commit --no-ff`) and let git tell you if it's clean.

D-05. Mirror Load Count Sheet colors exactly (not reinterpret)

Context. Team's spreadsheet "Color Key" uses saturated Google Sheets defaults (pure green, cyan, magenta, yellow). Those colors would look jarring against Jared's cobalt + cream UI palette.

Options weighed.

1. Use same hex values exactly – zero learning curve, clashes visually
2. Use same hue family but desaturate – cleaner UI, less 1:1 recognition
3. Hybrid – left-edge stripe in saturated color + pill with luminance- adjusted text, everything else uses refined palette

Chose option 3. Team gets instant visual recognition from the stripe + pill (they recognize cyan = missing driver before reading). The rest of the UI stays refined. Later added `pillTextColor` with Rec. 601 luminance so yellow/cyan/green pills render in near-black and purple/magenta in white – readability without color compromise.

Why it matters. "Match the spreadsheet exactly" and "look good as software" weren't actually in conflict once we found the hybrid. First- impulse options were both wrong in isolation; the right answer preserved both signals by scoping them differently (saturated for status identification, refined for everything else).

D-06. Validation page filters historical_complete server-side

Context. After raising ERA_CUTOFF, the validation queue summary still counted 28K+ pending assignments – because the query joined assignments without filtering the parent load's `historical_complete` flag. UI showed a big number that wasn't work-to-do.

Options weighed.

1. Filter on the frontend – client-side misses count, pagination breaks
2. Delete/cancel the 27K stale pending assignments – drastic, data loss
3. Filter in the service query at the join – returns correct counts

Chose option 3. Added `innerJoin(loads, ...)` + `eq(loads.historicalComplete, false)` to both the summary and tier queries. Validation queue fell 28K -> 2,939.

Why it matters. Data surface and data source should agree. If the number says "2,939 to validate", the table should show 2,939 items. Filtering at the service boundary (not the UI) is the only way to keep counts and content in sync.

D-07. Inline notes via assignments.notes (existing half-wired path)

Context. Jessica's repeated ask for inline comments on loads ("held for rate"). Discovered the assignments PUT route already accepted `notes` in its schema but the column didn't exist in the table.

Options weighed.

1. New `load_comments` table – threaded, multi-user, future-proof
2. Reuse breadcrumbs table with `event_type='load_note'` – no schema change
3. Add `notes text` column on assignments – simplest, uses existing half- built endpoint

Chose option 3. Migration 0008 added single column; existing PUT route started working immediately; ExpandDrawer got an editable textarea. Future multi-user comments can migrate to a dedicated table when threading becomes a real ask.

Why it matters. Half-built features in a codebase are often closer to shipping than starting fresh. The route already accepted `notes` because someone previously expected this would happen; completing the thought was 1 line of schema vs 1 day of new table+routes+UI.

D-08. Feedback loop via wells.matchFeedback (existing column)

Context. After shipping BOL manual-match UI, realized the loop was aspirational – manual matches didn't feed back into auto-matching. Validation page confirmations DID learn (via addWellAlias on location_mappings) but BOL reconciliation didn't.

Options weighed.

1. New `match_signals` table – clean data model, requires migration
2. Reuse `wells.matchFeedback` jsonb (already exists with 'manual_assign' action) – zero schema, minimal code
3. Machine-learning pipeline – overkill for current volume

Chose option 2. On manual match, append `{sourceName: driverName, action: 'manual_assign', by, at}` to the matched well's matchFeedback. In Tier 3 fuzzy disambiguation, query same jsonb to prefer wells with most historical confirmations for this driver. Ships compounding learning with no new tables.

Why it matters. Schema additions are expensive (migration risk, ORM regen, operational overhead). When existing columns can carry the signal with acceptable read/write patterns, use them. The `matchFeedback` jsonb was literally designed for this; the `manual_assign` action was already in the type union.

D-09. Port Python validators to TypeScript, not rebuild

Context. Need deterministic BOL format validation to gate Tier 1 matching and surface OCR-quality signals. Existing Python implementation in `jryan5150/bol-ocr-pipeline/lib/validators.py` is well-tested, domain- specific, production-ready.

Options weighed.

1. Rebuild from scratch in TS – risks behavior drift
2. Call out to Python subprocess – latency, deploy complexity
3. Direct TypeScript port preserving function signatures – future interop with Bryan's Jetson output, zero runtime cost

Chose option 3. Ported validateTicketNumber, validateWeight, validateNetTons, validateDate. Smoke-tested 11 edge cases (real BOLs pass, dates/times/addresses/short-tokens fail). Kept `FIELD_VALIDATORS` as an object mirroring the Python dict so drop-in when Jetson output lands.

Why it matters. When Bryan's Jetson ships a ready-to-consume extraction format, its fields will land in the same validator layer without re-work. Parallel implementations of the same logic in two languages stay in sync if the function surface stays identical; that's cheaper than a single "shared" implementation through some IPC boundary.

D-10. Two-tile photo source strategy (JotForm live, Jetson future)

Context. Photo pipeline has two sources: JotForm (live, working today) and Jetson (Bryan's offline pipeline, TBD). JotForm sync had stalled 12 days because it wasn't on cron.

Options weighed.

1. Wait for Jetson, then build pipeline – loses 12 days of photos, makes the demo today look broken
2. Stand JotForm back up + add to cron + treat as primary – ships value immediately, Jetson becomes additive
3. Build unified ingest that accepts either source – over-engineered for current scope

Chose option 2. Scheduled JotForm to run every 30 min, triggered manual catch-up sync (+1,212 submissions in 24h), wired match -> assignment. photo_status. Jetson endpoint (Task #17) is planned but not blocking; it'll feed the same matcher when sample arrives.

Why it matters. The tempting move when a new source is coming is to wait and build once. The right move is usually to make the current source first-class, prove the pipeline, and add new sources as they arrive. Otherwise you're betting the demo on a vendor's timeline.

D-11. Mike's expenses question is not a dollars conversation

Context. Mid-afternoon, Jessica said Mike is "looking at expenses and making decisions." Tempting to treat as a pricing crisis.

Options weighed.

1. Prepare defense of the billing – rate cards, hours tracked, etc.
2. Offer scope reduction – lower monthly cost
3. Reframe as a timeline conversation – the PCS OAuth blocker is what's making the engagement feel expensive, not the actual price

Chose option 3 (tentative – to be confirmed tomorrow). The engagement billed ~$5,500 vs market rate of $25-50k+. Client isn't overpaying; they're frustrated that value hasn't yet translated to labor savings (because PCS typing is unchanged until OAuth lands). The right conversation is "here's what's shipped, here's what's blocked on your vendor, here's what changes when they unblock."

Why it matters. "Expenses review" sounds like a dollars problem and usually isn't. It's almost always a value-per-dollar problem, which is really a visibility/timeline problem. Fix the visibility, the dollars question evaporates. (This is an active hypothesis; validated or invalidated in tomorrow's 10:30 call.)

Decisions are the portable part. Commits age; tables grow; UI changes. Why something was built the way it was built is what lets the next engineer make the next decision correctly.

ES Express Dispatch Team Feedback Ledger

Generated: 2026-04-14 **Last updated:** 2026-04-14 (post-ship batch – 13 additional items landed) **Sources:**

- `docs/transcripts/2026-04-06-validation-walkthrough.md` (Jessica, 1h 14m)
- `docs/transcripts/2026-04-09-team-follow-up.md` (full team, 1h 2m)
- `docs/2026-04-06-round-2-consolidation.md` (38 issues)

- `docs/2026-04-06-*-round2-report.md` (per-persona reports)
- `docs/2026-04-06-validation-call-findings.md`
- `git log main --oneline` (ship verification)

Executive Summary

- **Total asks captured:** 52 (38 Round-2 + 14 new from transcripts)
- **Shipped (verified via git + code):** 43 (83%)
- **Partial / in-progress:** 0
- **Open:** 2 (4% – both large-effort, deferred-by-scope)
- **Blocked on external:** 2 (4%)

Items still open:

1. User invite / role / deactivate flow – admin, large
2. Architecture: BOL + Validation reconciliation feel decoupled – large

Blocked on external:

3. PCS auto-push integration + 3 PCS-dependent color statuses – waiting on Kyle's OAuth credentials
4. Jetson JSONL BOL import endpoint – waiting on sample record from Bryan
5. Keyboard navigation between loads – Stephanie, medium

SHIPPED (verified against git + live code)

#	Ask	Source	Commit	Verified
S-01	ES Express LLC logo on login + sidebar	Jared / Apr 9 40:31	`829eda0` (merged today)	[x] live
S-02	Cobalt primary + cream backgrounds (swap from purple)	Jared	`3aa3243` via merge `829eda0`	[x] live
S-03	Collapsible sidebar with admin grouping	Jared	`c85ca1f` via merge `829eda0`	[x] live
S-04	Load Count Sheet color coding on rows	Apr 9 16:23	`ad8b1d2`, `a72324d`	[x] today
S-05	Color Key legend popover	Apr 9 16:34	`ad8b1d2`	[x] today
S-06	Date range filter (from->to)	Apr 9, R2	`a72324d`	[x] today
S-07	Match audit UI on load drawer	Jessica validation call	`b4aa9f0`, `94a07a2`	[x]
S-08	Auto-mapper anti-hallucination rules	Validation findings	`e15a6e8`	[x]
S-09	Archive / historical complete banner	Apr 6 validation call	`79180a9`	[x]

#	Ask	Source	Commit	Verified
S-10	Historical-complete classifier on sync	Apr 6 "archive vs live"	`f50e97b`	[x]
S-11	Visible "Search BOL" button (M8)	Validation call 23:15	`de148d1`	[x]
S-12	Logistiq carrier export (no session auth)	Operational	`43e6b2a`	[x]
S-13	`weight_lbs` column + sync population	Round 4	`3d9a5bf`, `5d91282`	[x]
S-14	P0-3 single-load Validate confirm	Round 2	shipped (confirm pattern in DispatchDesk)	[x]
S-15	P0-4 Missing Ticket button – wired then disabled with tooltip	R2 -> R3 -> R4	`6e506ab` -> `6ea015a`	[x] current state
S-16	P2-6 Advance All confirmation	Round 2	`53a5eea`	[x]
S-17	P2-8 Pagination reset on filter change	Round 2	`e0439d8`	[x]
S-18	P1-4 onClaim in Loads view	Round 2	`b8ff924`	[x]
S-19	P1-12 "Entered" -> "Mark Entered"	Round 2	`22cfec4`	[x]
S-20	P2-1 Demurrage panel recolor	Round 2	`5d37e56`	[x]
S-21	P2-7 Jargon tooltips (BOL, FSC, Reconciled, Tiers)	Round 2 / Katie	`9ac11aa`	[x]
S-22	P1-11 prefers-reduced-motion animation cap	Round 2	`d1ea180`	[x]
S-23	P2-3 "Last updated" pill on home Objectives	Round 2	`b39dee3`	[x]
S-24	P1-8 Companies admin behind feature flag	Round 2	`902b39e` + revert `ec4fb64`	[x] current: visible
S-25	Pill text readability per-hex luminance	Apr 14 team note	`a72324d`	[x] today
S-26	enteredIds persists across page refresh (P1-5)	Stephanie R2	`feae3c9`	[x] today
S-27	LoadRow checkbox keyboard accessible (P1-3)	R2	`feae3c9`	[x] today
S-28	Loads view row click expands drawer in place (P0-6)	Stephanie R2	`feae3c9`	[x] today
S-29	PhotoModal "Flag for Jessica" wired (P1-6)	R2	`feae3c9`	[x] today
S-30	WellsAdmin pencil edits dailyTargetLoads (P0-1)	Admin R2	`feae3c9`	[x] today

#	Ask	Source	Commit	Verified
S-31	WellsAdmin defaults to Active filter (P2-9)	Admin R2	`feae3c9`	[x] today
S-32	WellsAdmin paginates 25/page (P2-10)	Admin R2	`feae3c9`	[x] today
S-33	PCS numbering correct under filters (P1-7)	Stephanie R2	`feae3c9`	[x] today
S-34	PropX `staging_*` schema-drift fields known	Apr 14 sync log	`feae3c9`	[x] today
S-35	Copy ready-loads-as-TSV report (P2-2)	Jessica R2	`feae3c9`	[x] today
S-36	Home "Confirmed & Ready" card clickable (P1-10)	Stephanie R2	`feae3c9`	[x] today
S-37	Inline editable assignment notes (O-05)	Jessica Apr 6	`ef40761` + migration `0008`	[x] today
S-38	Keyboard navigation between rows (j/k///Enter/Esc)	Stephanie R1 + Apr 9	`ef40761`	[x] today
S-39	Validation page: more inline edits + pencil affordance (P2-4)	R2	`29996ae`	[x] today
S-40	Photo gate on dispatch_ready transition (P-03)	Jessica Apr 6 ~54:20	`d6d979c`	[x] today
S-41	Need Well Rate Info status wired (O-23) – admin checkbox + 6th color now live	Apr 9 + admin	`d6d979c` + migration `0009`	[x] today
S-42	Missed Loads Report – pull-style page Jessica runs on demand (O-16)	Jessica Apr 6	`d6d979c`	[x] today
S-43	Mark Entered button now in ExpandDrawer (P-01)	Stephanie R2 (P1-2)	`d6d979c`	[x] today

PARTIAL (in-progress / partly-addressed)

#	Ask	Source	What's done	What's missing	Effort
P-01	Mark Entered control in ExpandDrawer	Stephanie R2 (P1-2)	Button exists in LoadRow only	Add to drawer – dispatchers open drawer more than they row-click	small
P-02	Contrast / readability pass	Jared + Apr 9 team	Jared's pass + pill contrast today	Full a11y audit with team review	medium
P-03	Photo requirement in "100% ready" logic	Jessica Apr 6	`isTier1 = hasAllFieldMatches && hasPhoto` rule exists in match-rules	Not yet applied to auto-promote logic – Tier 1 still allowed without photo in some paths	small

OPEN (ready to work on)

All "Now" + most "This week" items shipped 2026-04-14. Remaining open items are large-effort (>=4 hrs) or explicitly deferred.

#	Ask	Source	File hint	Effort
O-12	P0-5 User invite / role / deactivate flow absent	Admin R2	`auth/routes.ts` + `UsersAdmin.tsx`, `invite.ts` orphaned	large
O-13	P1-9 Admin pages client-side role-gated	Admin R2	**explicitly skipped per `project_admin_rbac_intentional.md`** (all users admin during rollout)	– skip
O-24	BOL + Validation reconciliation feel decoupled (architecture)	Apr 6 audit	larger refactor surfaced when discussing how matching informs validation; not blocking but worth re-think	large

Post-PCS (needs OAuth credentials from Kyle)

#	Ask	Source	Blocking	Effort after unblock
O-18	"Validate" -> "Push to PCS" button rename	Apr 9 25:00	PCS OAuth	trivial
O-19	Push-to-PCS actual integration (vs copy/paste)	Jessica Apr 9	PCS OAuth + Kyle	large
O-20	"Loads Being cleared" / "Loads cleared" status colors wired (2 of 9)	Apr 9	PCS OAuth (observability)	small
O-21	"Export (Transfers) Completed" status wired	Apr 9	PropX export API spec	small
O-22	"Invoiced" status wired	Apr 9	Finance module integration	small
O-23	"Need Well Rate Info" status wired	Apr 9	New `well.rate_status` column + admin UI	small

BLOCKED (external)

#	Ask	Dependency	Owner
B-01	Jetson JSONL BOL import endpoint (Task #17)	Sample output record from Bryan's Jetson	Bryan / Jetson output
B-02	PCS OAuth credentials for all `O-18` through `O-20` items above	Kyle / PCS vendor	Kyle

---

Appendix A – NEW items from Apr 6 + Apr 9 transcripts (not in Round 2 consolidation)

These surfaced in the recordings but weren't captured in the written Round 2 docs. All tracked above in the OPEN section unless noted.

Apr 6 Validation Walkthrough		
Inline comments on loads	21:45-ish	O-05
Photo requirement strict enforcement	54:20	P-03
Missed-loads on-demand report	~48:00	O-16
Live data before finalizing workflow	entire session	covered by today's migration

Apr 9 Team Follow-up		
Load Count Sheet color scheme -> UI	16:23-16:34	S-04, S-05 [x] shipped today
Date range lookup	~18:00	S-06 [x] shipped today
"Validate" button will need rename when PCS lands	25:00-27:02	O-18 deferred
Red flag on ticket mismatches (PropX-like)	27:20	partly covered by match_audit reasons (S-07)
"Hard to visualize without live data"	Scout 35:00	resolved today (real DB live)
Jared contrast + logo changes	40:31	S-01, S-02, S-03

Inconsistencies resolved

Round 2 consolidation listed some items as open that git shows as shipped. I verified each and moved them to SHIPPED above: P2-6, P2-3, P1-12, P1-11, P2-7, P2-1, P2-8, P1-4.

Round 2 P0-4 (Missing Ticket) was shipped, then intentionally reverted to disabled-with-tooltip per commit `6ea015a`. Current state is the intended state.

Round 2 P0-2 (Delivered Date inline-edit broken schema) is fixed – `backend/src/plugins/dispatch/routes/loads` now declares `deliveredOn`.

Round 2 P1-1 (`capitalize` mangling) is fixed in `FilterTabs.tsx` (removed in Round 3). Leftover `capitalize` in `BolQueue.tsx:206` was benign (labels already cased) – removed 2026-04-14 for hygiene.

Maintained alongside session hand-offs. Update after each validation session or ship event.